SOCKET PROGRAMMING

SERVER:

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <winsock2.h>

#pragma comment(lib, "ws2_32.lib")

int main() {

WSADATA wsa;

SOCKET server, client;

struct sockaddr_in serverAddr, clientAddr;

int c;

char buffer[1024];

WSAStartup(MAKEWORD(2,2), &wsa);

server = socket(AF_INET, SOCK_STREAM, 0);

serverAddr.sin_family = AF_INET;

serverAddr.sin_addr.s_addr = INADDR_ANY;

serverAddr.sin_port = htons(8080);

bind(server, (struct sockaddr*)&serverAddr, sizeof(serverAddr));

listen(server, 3);

printf("Server waiting for connection...\n");

c = sizeof(struct sockaddr_in);

client = accept(server, (struct sockaddr*)&clientAddr, &c);

recv(client, buffer, sizeof(buffer), 0);

printf("Received: %s\n", buffer);

strcpy(buffer, "Message received successfully");
```

```
send(client, buffer, strlen(buffer), 0);

closesocket(client);

closesocket(server);

WSACleanup();

return 0;

}
```

CLIENT:

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <winsock2.h>

#pragma comment(lib, "ws2_32.lib")

int main() {

WSADATA wsa;

SOCKET sock;

struct sockaddr_in server;

char message[] = "Hello from Client";

char buffer[1024];

WSAStartup(MAKEWORD(2,2), &wsa);

sock = socket(AF_INET, SOCK_STREAM, 0);

server.sin_family = AF_INET;

server.sin_addr.s_addr = inet_addr("192.168.1.10"); // SERVER IP

server.sin_port = htons(8080);

connect(sock, (struct sockaddr*)&server, sizeof(server));

send(sock, message, strlen(message), 0);

recv(sock, buffer, sizeof(buffer), 0);
```

```
printf("Server reply: %s\n", buffer);

closesocket(sock);

WSACleanup();

return 0;

}
```

Guest to Guest

Server.c

```
#include <stdio.h>
#include <string.h>
#include <arpa/inet.h>
#include <unistd.h>
int main() {
int s, c;
struct sockaddr_in server, client;
char msg[100];
s = socket(AF_INET, SOCK_STREAM, 0);
server.sin_family = AF_INET;
server.sin_addr.s_addr = INADDR_ANY;
server.sin_port = htons(8080);
bind(s, (struct sockaddr*)&server, sizeof(server));
listen(s, 1);
printf("Waiting...\n");
c = accept(s, NULL, NULL);
read(c, msg, sizeof(msg));
printf("Client: %s\n", msg);
send(c, "Hello Client", 12, 0);
close(s);
}
```

Client.c

```
#include <stdio.h>
#include <string.h>
#include <arpa/inet.h>
#include <unistd.h>
int main() {
int s;
struct sockaddr_in server;
char msg[100];
s = socket(AF_INET, SOCK_STREAM, 0);
server.sin_family = AF_INET;
server.sin_port = htons(8080);
inet_pton(AF_INET, "127.0.0.1", &server.sin_addr);
connect(s, (struct sockaddr*)&server, sizeof(server));
```

```
send(s, "Hello Server", 12, 0);
read(s, msg, sizeof(msg));
printf("Server: %s\n", msg);
close(s);
}
```

```
gcc server.c -o server
gcc client.c -o client
```

IPC

Unidir

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>
int main() {
int fd[2];
char buffer[100];
pipe(fd);
if (fork() == 0) {
// Child process
close(fd[1]); // close write end
read(fd[0], buffer, sizeof(buffer));
printf("Child received: %s\n", buffer);
close(fd[0]);
} else {
// Parent process
close(fd[0]); // close read end
char msg[] = "Hello Child";
write(fd[1], msg, strlen(msg)+1);
close(fd[1]);
}
return 0;
}
```

BIDIR

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>
int main() {
int pipe1[2], pipe2[2];
char buffer[100];
pipe(pipe1);
pipe(pipe2);
if (fork() == 0) {
```

```
// Child
close(pipe1[1]);
read(pipe1[0], buffer, sizeof(buffer));
printf("Child received: %s\n", buffer);
close(pipe2[0]);
char msg[] = "Hello Parent";
write(pipe2[1], msg, strlen(msg)+1);
} else {
// Parent
close(pipe1[0]);
char msg[] = "Hello Child";
write(pipe1[1], msg, strlen(msg)+1);
close(pipe2[1]);
read(pipe2[0], buffer, sizeof(buffer));
printf("Parent received: %s\n", buffer);
}
return 0;
}
```

RMI:

Calculate.java

```
import java.rmi.Remote;

import java.rmi.RemoteException;

public interface Calculate extends Remote {

String getTable(int number) throws RemoteException;
}
```

Server.java

```
import java.rmi.registry.LocateRegistry;

import java.rmi.registry.Registry;

import java.rmi.server.UnicastRemoteObject;

public class Server extends UnicastRemoteObject implements Calculate {

// Constructor

protected Server() throws java.rmi.RemoteException {

super();

}

// Remote Method
```

```
@Override
public String getTable(int number) {
String result = "";
for(int i = 1; i <= 10; i++) {
result += number + " x " + i + " = " + (number * i) + "\n";
}
return result;
}
public static void main(String[] args) {
try {
Registry registry = LocateRegistry.createRegistry(1099);
Server server = new Server();
registry.rebind("TableService", server);
System.out.println("RMI Server Started...");
}
catch(Exception e) {
e.printStackTrace();
}
}
}
```

Client.java

```
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;
import java.util.Scanner;
public class Client {
public static void main(String[] args) {
try {
Registry registry = LocateRegistry.getRegistry("localhost", 1099);
```

```
Calculate stub = (Calculate) registry.lookup("TableService");
Scanner sc = new Scanner(System.in);
System.out.print("Enter number: ");
int num = sc.nextInt();
String table = stub.getTable(num);
System.out.println("\nMultiplication Table:");
System.out.println(table);
sc.close();
}
catch(Exception e) {
e.printStackTrace();
}
}
}
```

CLOCK

LogicalClock

```
import java.util.Scanner;
class LogicalClock {
int[][] clock;
int processes, events;
LogicalClock(int p, int e) {
processes = p;
events = e;
clock = new int[p][e];
}
void initializeClock() {
```

```
for (int i = 0; i < processes; i++) {
for (int j = 0; j < events; j++) {
clock[i][j] = j + 1;
}
}
}
void updateMessage(int sp, int se, int rp, int re) {
clock[rp][re] = Math.max(clock[rp][re], clock[sp][se]) + 1;
for (int i = re + 1; i < events; i++) {
clock[rp][i] = clock[rp][i - 1] + 1;
}
}
void displayClock() {
for (int i = 0; i < processes; i++) {
for (int j = 0; j < events; j++) {
System.out.print(clock[i][j] + " ");
}
System.out.println();
}
}
int getClockValue(int p, int e) {
return clock[p][e];
}
}
```

Vector Clock

```
class VectorClock {
```

```
int[][][] clock;
int processes, events;
VectorClock(int p, int e) {
processes = p;
events = e;
clock = new int[p][e][p];
}
void initializeVectorClock() {
for (int i = 0; i < processes; i++) {
for (int j = 0; j < events; j++) {
for (int k = 0; k < processes; k++) {
clock[i][j][k] = 0;
}
clock[i][j][i] = j + 1;
}
}
}
void updateMessage(int sp, int se, int rp, int re) {
for (int i = 0; i < processes; i++) {
clock[rp][re][i] = Math.max(clock[rp][re][i], clock[sp][se][i]);
}
clock[rp][re][rp]++;
for (int j = re + 1; j < events; j++) {
for (int i = 0; i < processes; i++) {
clock[rp][j][i] = clock[rp][j - 1][i];
}
```

```
clock[rp][j][rp]++;
}
}
void displayVectorClock() {
for (int i = 0; i < processes; i++) {
for (int j = 0; j < events; j++) {
System.out.print("[");
for (int k = 0; k < processes; k++) {
System.out.print(clock[i][j][k] + " ");
}
System.out.print("] ");
}
System.out.println();
}
}
int[] getVector(int p, int e) {
return clock[p][e];
}
}
```

Clock Simulation:

```
import java.util.Scanner;
public class ClockSimulation {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
System.out.print("Enter number of processes: ");
int p = sc.nextInt();
```

```
System.out.print("Enter number of events per process: ");
int e = sc.nextInt();
LogicalClock lc = new LogicalClock(p, e);
lc.initializeClock();
VectorClock vc = new VectorClock(p, e);
vc.initializeVectorClock();
System.out.print("Enter number of message events: ");
int m = sc.nextInt();
for (int i = 0; i < m; i++) {
System.out.println("Message " + (i + 1));
System.out.print("Sender process index: ");
int sp = sc.nextInt();
System.out.print("Sender event index: ");
int se = sc.nextInt();
System.out.print("Receiver process index: ");
int rp = sc.nextInt();
System.out.print("Receiver event index: ");
int re = sc.nextInt();
lc.updateMessage(sp, se, rp, re);
vc.updateMessage(sp, se, rp, re);
}
System.out.println("\nLogical Clock Values:");
lc.displayClock();
System.out.println("\nVector Clock Values:");
vc.displayVectorClock();
sc.close();
```

```
}

}
```

OPENMP:

```
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>
#define N 3 // Matrix size (NxN)
int main()
{
int i, j, k;
int A[N][N], B[N][N], C[N][N];
int global_count = 0;
printf("Matrix size: %d x %d\n", N, N);
printf("\nInitializing matrices...\n");
for(i = 0; i < N; i++)
for(j = 0; j < N; j++)
{
A[i][j] = i + j;
B[i][j] = i * j;
C[i][j] = 0;
}
omp_set_dynamic(0);
omp_set_num_threads(4);
printf("\nMaximum threads available: %d\n", omp_get_max_threads());
#pragma omp parallel private(i,j,k) shared(A,B,C) reduction(+:global_count)
{
int tid = omp_get_thread_num();
#pragma omp single
printf("\nTotal threads working: %d\n", omp_get_num_threads());
#pragma omp for schedule(dynamic)
for(i = 0; i < N; i++)
{
for(j = 0; j < N; j++)
{
for(k = 0; k < N; k++)
{
C[i][j] += A[i][k] * B[k][j];
global_count++;
}
}
printf("Thread %d completed row %d\n", tid, i);
}
}
printf("\nResult Matrix C:\n");
for(i = 0; i < N; i++)
{
for(j = 0; j < N; j++)
printf("%4d ", C[i][j]);
printf("\n");
}
```

```
printf("\nTotal multiplication operations = %d\n", global_count);
return 0;
}
```

```
gcc openmp.c -o openmp -fopenmp
```

MPI

```
#include "mpi.h"
#include <stdio.h>

int main(int argc, char *argv[])
{
   int data[] = {1, 2, 3, 4, 5, 6, 7}; // Array size >= no. of processes
   int rank, size;
   int i, j;

   MPI_Init(&argc, &argv);
   MPI_Comm_rank(MPI_COMM_WORLD, &rank);
   MPI_Comm_size(MPI_COMM_WORLD, &size);
   MPI_Scatter(data, 1, MPI_INT, &i, 1, MPI_INT, 0, MPI_COMM_WORLD);

   printf("Process %d received value %d\n", rank, i);
   MPI_Reduce(&i, &j, 1, MPI_INT, MPI_PROD, 0, MPI_COMM_WORLD);
   if(rank == 0)
   {
      printf("\nFinal Product = %d\n", j);
   }

   MPI_Finalize();
   return 0;
}
```

```
mpicc program.c -o program

mpirun -np 4 ./program
```

RICART AGRAWALA

```
import java.util.*;
```

```
enum ProcessState {
RELEASED, WANTED, HELD
}
class Process extends Thread {
int id;
int clock = 0;
int requestTime;
ProcessState state = ProcessState.RELEASED;
int totalProcesses;
int replyCount = 0;
List<Integer> deferred = new ArrayList<>();
static Process[] processes;
Random rand = new Random();
Process(int id, int totalProcesses) {
this.id = id;
this.totalProcesses = totalProcesses;
}
void requestCS() {
synchronized (this) {
clock++;
state = ProcessState.WANTED;
requestTime = clock;
replyCount = 0;
System.out.println("\nProcess " + id + " requesting CS at time " + requestTime);
}
for (Process p : processes) {
```

```
if (p.id != this.id) {
new Thread(() -> {
try { Thread.sleep(rand.nextInt(500)); } catch (Exception e) {}
p.receiveRequest(this.id, this.requestTime);
}).start();
}
}
}
synchronized void receiveRequest(int senderId, int senderTime) {
clock = Math.max(clock, senderTime) + 1;
boolean sendReplyNow = false;
if (state == ProcessState.RELEASED) {
sendReplyNow = true;
}
else if (state == ProcessState.WANTED) {
if (senderTime < requestTime ||
(senderTime == requestTime && senderId < id)) {
sendReplyNow = true;
}
}
if (sendReplyNow) {
sendReply(senderId);
} else {
deferred.add(senderId);
}
}
```

```
void sendReply(int receiverId) {
new Thread(() -> {
try { Thread.sleep(rand.nextInt(500)); } catch (Exception e) {}
System.out.println("Process " + id + " sending REPLY to " + receiverId);
processes[receiverId].receiveReply(this.id);
}).start();
}
synchronized void receiveReply(int senderId) {
replyCount++;
System.out.println("Process " + id + " received REPLY from " + senderId);
if (replyCount == totalProcesses - 1) {
enterCS();
}
}
void enterCS() {
synchronized (this) {
state = ProcessState.HELD;
System.out.println("Process " + id + " ENTERING Critical Section");
}
try { Thread.sleep(1000); } catch (Exception e) {}
exitCS();
}
void exitCS() {
synchronized (this) {
state = ProcessState.RELEASED;
System.out.println("Process " + id + " EXITING Critical Section");
```

```
for (int pid : deferred) {
sendReply(pid);
}
deferred.clear();
}
}
public void run() {
try {
Thread.sleep(rand.nextInt(1000));
} catch (Exception e) {}
requestCS();
}
}
public class RicartAgarwalaDemo {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
System.out.print("Enter number of processes: ");
int n = sc.nextInt();
Process.processes = new Process[n];
for (int i = 0; i < n; i++) {
Process.processes[i] = new Process(i, n);
}
System.out.print("Enter number of processes requesting CS simultaneously: ");
int requests = sc.nextInt();
Set<Integer> selected = new HashSet<>();
for (int i = 0; i < requests; i++) {
```

```
System.out.print("Enter process ID (0 to " + (n-1) + "): ");
int pid = sc.nextInt();
if (pid >= 0 && pid < n) {
selected.add(pid);
} else {
System.out.println("Invalid ID");
i--;
}
}
for (int pid : selected) {
Process.processes[pid].start();
}
sc.close();
}
}
```

SUZUKI KASAMI

```
import java.util.*;
class Token {
    int[] LN;
    Queue<Integer> Q;

    public Token(int n) {
        LN = new int[n];
        Q = new LinkedList<>();
    }
```

```
    public void printToken(int siteId) {
        System.out.println("TOKEN at Site " + siteId);
        System.out.println("LN: " + Arrays.toString(LN));
        System.out.println("Queue: " + Q);
    }
}

class Site extends Thread {

    int id;
    int n;
    int[] RN;
    boolean hasToken = false;

    static Token token;
    static Site[] sites;

    public Site(int id, int n) {
        this.id = id;
        this.n = n;
        RN = new int[n];
    }

    // Print RN
    public void printRN() {
        System.out.println("Site " + id + " RN: " + Arrays.toString(RN));
    }
```

```
// ================= REQUEST CS =================
public void requestCS() {
    synchronized (Site.class) {
        RN[id]++;
        System.out.println("\n[REQUEST] Site " + id);
        printRN();

        for (int i = 0; i < n; i++) {
            if (i != id) {
                sites[i].receiveRequest(id, RN[id]);
            }
        }

        if (hasToken) {
            executeCS();
        }
    }
}

// ================= RECEIVE REQUEST =================
public synchronized void receiveRequest(int i, int sn) {
    RN[i] = Math.max(RN[i], sn);

    System.out.println("[RECEIVE REQUEST] Site " + id + " from " + i);
    printRN();

    if (hasToken && RN[i] == token.LN[i] + 1) {
```

```
        hasToken = false;
        System.out.println("[TOKEN SENT] " + id + " → " + i);
        sites[i].receiveToken(token);
    }
}

// ================= RECEIVE TOKEN =================
public synchronized void receiveToken(Token t) {
    token = t;
    hasToken = true;

    System.out.println("\n[RECEIVE TOKEN] Site " + id);
    token.printToken(id);

    executeCS();
}

// ================= EXECUTE CS =================
public void executeCS() {
    System.out.println("\n>>> Site " + id + " ENTERING CS");

    try {
        Thread.sleep(1000);
    } catch (Exception e) {}

    releaseCS();
}
```

```
// ================= RELEASE CS =================
public synchronized void releaseCS() {
    System.out.println("<<< Site " + id + " EXITING CS");

    // Update LN
    token.LN[id] = RN[id];
    System.out.println("[LN UPDATED]");
    token.printToken(id);

    // Update Queue
    for (int j = 0; j < n; j++) {
        if (j != id && !token.Q.contains(j) && RN[j] == token.LN[j] + 1) {
            token.Q.add(j);
            System.out.println("[QUEUE ADD] " + j);
            token.printToken(id);
        }
    }

    // Send token
    if (!token.Q.isEmpty()) {
        int next = token.Q.poll();
        hasToken = false;

        System.out.println("[TOKEN TRANSFER] " + id + " → " + next);
        sites[next].receiveToken(token);
    } else {
        System.out.println("[TOKEN IDLE at Site " + id + "]");
    }
```

```
    }

    public void run() {
        requestCS();
    }
}

public class SuzukiKasamiFinal {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of sites: ");
        int n = sc.nextInt();

        Site.sites = new Site[n];

        for (int i = 0; i < n; i++) {
            Site.sites[i] = new Site(i, n);
        }

        // Initial token at Site 0
        Site.token = new Token(n);
        Site.sites[0].hasToken = true;

        System.out.println("\nInitial token at Site 0");
        Site.token.printToken(0);
```

```
        System.out.print("\nEnter number of requests: ");
        int r = sc.nextInt();

        System.out.println("Enter site IDs:");

        List<Thread> threads = new ArrayList<>();

        for (int i = 0; i < r; i++) {
            int id = sc.nextInt();
            threads.add(new Thread(Site.sites[id]));
        }

        // Start all threads (parallel requests)
        for (Thread t : threads) {
            t.start();
        }

        // Wait
        for (Thread t : threads) {
            try {
                t.join();
            } catch (Exception e) {}
        }

        sc.close();
    }
}
```

CHANDY AND

```
#include <stdio.h>

int n;
int graph[10][10];
int visited[10], stack[10];

int hasCycle(int v) {
    visited[v] = 1;
    stack[v] = 1;

    for (int i = 0; i < n; i++) {
        if (graph[v][i]) {
            if (!visited[i] && hasCycle(i))
                return 1;
            else if (stack[i])
                return 1;
        }
    }

    stack[v] = 0;
    return 0;
}

int main() {
    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter wait-for graph (adjacency matrix):\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &graph[i][j]);

    for (int i = 0; i < n; i++) {
        if (!visited[i] && hasCycle(i)) {
            printf("Deadlock Detected (Cycle exists)\n");
            return 0;
        }
    }

    printf("No Deadlock\n");
    return 0;
}
```

CHANDY OR

```
#include <stdio.h>

int main() {
    int n;
    int graph[10][10];
    int blocked = 1;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter dependency matrix:\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &graph[i][j]);

    for (int i = 0; i < n; i++) {
        int hasOption = 0;

        for (int j = 0; j < n; j++) {
            if (graph[i][j] == 0) {
                hasOption = 1;
                break;
            }
        }

        if (hasOption) {
            blocked = 0;
            break;
        }
    }

    if (blocked)
        printf("Deadlock Detected (Knot)\n");
    else
        printf("No Deadlock\n");

    return 0;
}
```

RPC Implementation Matrix Addition:

matrix.x

```
/* matrix.x */

typedef int row[10];
```

```
struct matrix {
    int row_count;
    int col_count;
    row data[10];
};

struct mat_pair {
    matrix m1;
    matrix m2;
};

program MATRIX_PROG {
    version MATRIX_VERS {
        matrix ADD(mat_pair) = 1;
    } = 1;
} = 0x20000001;
```

In Terminal run: rpcgen matrix.x

matrix_server.c:

```
#include "matrix.h"
#include <stdio.h>

matrix *
add_1_svc(mat_pair *argp, struct svc_req *rqstp)
{
    static matrix result;
    int i, j;

    result.row_count = argp->m1.row_count;
    result.col_count = argp->m1.col_count;

    for (i = 0; i < result.row_count; i++) {
        for (j = 0; j < result.col_count; j++) {
            result.data[i][j] =
                argp->m1.data[i][j] + argp->m2.data[i][j];
        }
    }
    return &result;
}
```

matrix_client.c

```
#include "matrix.h"
#include <stdio.h>

int main(int argc, char *argv[])
{
```

```
CLIENT *cl;
matrix m1, m2;
mat_pair args;
matrix *res;
int i, j;

if (argc < 2) {
    printf("Usage: %s server_ip\n", argv[0]);
    return 1;
}

cl = clnt_create(argv[1], MATRIX_PROG, MATRIX_VERS, "tcp");
if (cl == NULL) {
    clnt_pcreateerror(argv[1]);
    return 1;
}

m1.row_count = m2.row_count = 2;
m1.col_count = m2.col_count = 2;

printf("Enter Matrix 1:\n");
for (i = 0; i < 2; i++)
    for (j = 0; j < 2; j++)
        scanf("%d", &m1.data[i][j]);

printf("Enter Matrix 2:\n");
for (i = 0; i < 2; i++)
    for (j = 0; j < 2; j++)
        scanf("%d", &m2.data[i][j]);

args.m1 = m1;
args.m2 = m2;

res = add_1(&args, cl);
if (res == NULL) {
    clnt_perror(cl, "RPC failed");
    return 1;
}

printf("Result Matrix:\n");
for (i = 0; i < res->row_count; i++) {
    for (j = 0; j < res->col_count; j++)
        printf("%d ", res->data[i][j]);
    printf("\n");
}

clnt_destroy(cl);
return 0;
```

}

In Terminal:
sudo apt update

sudo apt install libtirpc-dev rpcbind

gcc matrix_server.c matrix_svc.c matrix_xdr.c \
-I/usr/include/tirpc -ltirpc -o server

gcc matrix_client.c matrix_clnt.c matrix_xdr.c \
-I/usr/include/tirpc -ltirpc -o client

sudo service rpcbind start
To check: rpcinfo -p

Terminal 1: ./server
Terminal 2: ./client localhost